

DLIC Homework 4

In-System Debugging Using ILA core

系級:資訊 114

學號:E34106010

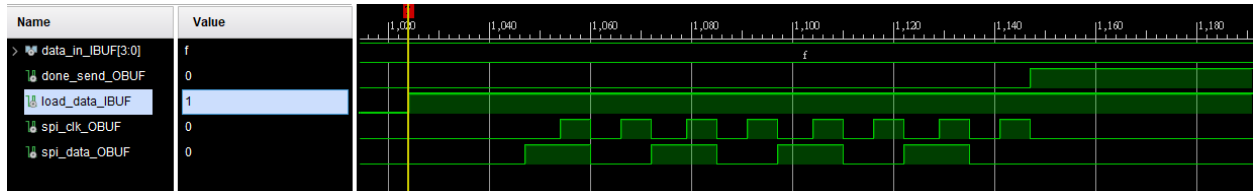
姓名:黃偉峰

Part1: SPI controller 設計說明

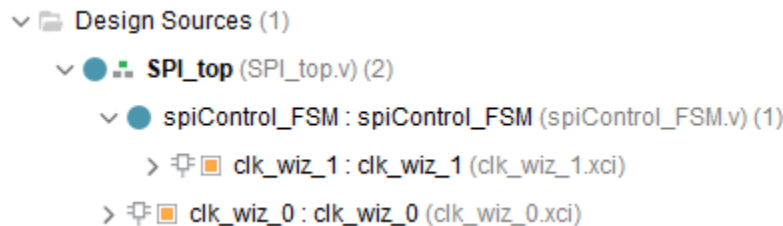
以下表格為 SPI 的四種 Mode，此次實作我以 Mode 0 來當作實作 SPI master 端的標準。

SPI mode	CPOL (Clock Polarity)	CPHA (Clock Phase)	SCLK 閒置電平	資料在哪個 clk edge 被擷取 (接收端)	資料在哪個 clk edge 設置 (傳送端)
Mode 0	0	0	Low	第一個 rising edge	Falling edge
Mode 1	0	1	Low	第二個 falling edge	Rising edge
Mode 2	1	0	High	第一個 falling edge	Rising edge
Mode 3	1	1	High	第二個 rising edge	Falling edge

在 Mode 0 的規定下，SPI master 在閒置時的 SPI_CLK 必須為 Low，並且在傳輸資料的時候，Master 端需要在 SPI_CLK 的 Negative Edge 時設置資料，讓 Receiver 在第一個 SPI_CLK 為 Positive Edge 時就可以讀取資料，下方為傳輸過程的波形圖：



從上圖可以看到 spi_clk 為 Master 傳輸到 Receiver 的同步傳輸 Clock，load_data 被拉高後把 data 輸入進 Master 端並進行傳輸到 Receiver 的動作，spi_data 第一筆資料在 spi_clk 第一個 Rising Edge 抵達前就已設定完成，使得 Receiver 可以在 spi_clk Rising Edge 時可以讀取到正確的資料，並且 spi_data 在 spi_clk 的 Falling Edge 時設定新的要傳輸的資料。



上圖為本次設計的 Hierarchy，spiControl_FSM 主要為 SPI master 的電路實作內容，SPI_top 則為將 clk_wizard 和 spiControl_FSM 包裝起來的 Top Module。

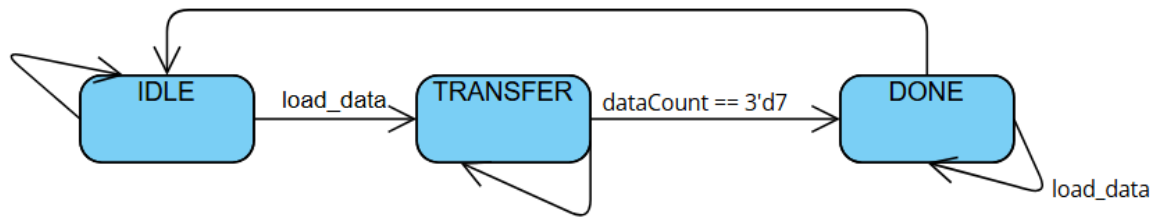
因本次實作助教提供的作業 SPI_controller module input clock 需求為 100MHz，然後 Pynq 板上所提供的 free running clock 為 125MHz，必須先將 125MHz 的 native clock 轉換成 100MHz，這邊設計上我直接使用 xilinx 提供的 clock wizard 來作 Frequency Divider。下圖為 SPI_top 的程式碼，主要是將 125MHz 的 clock 轉換成 100MHz 的頻率提供給 spiControl_FSM 使用。

```
module SPI_top(  
  ///input  
  input clk, //On-board pynq clk 125MHz (H16 pin)  
  input rst,  
  input [3:0] data_in,  
  input load_data,  
  ///output  
  output spi_clk, //spi_clk output 10MHz  
  output spi_data,  
  output done_send  
);  
  
  wire clk_100MHz;  
  
  spiControl_FSM(.clk(clk_100MHz), .rst(rst), .data_in(data_in), .load_data(load_data), .spi_clk(spi_clk), .spi_data(spi_data),  
  clk_wiz_0(.clk_in1(clk), .reset(rst), .clk_out1(clk_100MHz));  
  
endmodule
```

在 spiControl_FSM 當中需要做到將 input clk 100MHz 轉換到 output spi_clk 10MHz，因此我這邊一樣使用 xilinx 內建的 clock wizard 來作 Frequency Divder，使用 xilinx 提供的 clock_wizard IP 可以讓 FPGA 使用 MMCM 合成出穩定可靠的 clock 輸出端會比我們自己寫的 Frequency Divider 來的好。下圖為 spiControl_FSM 使用 clock_wizard 的部分。

```
module spiControl_FSM(  
  ///input  
  input clk, //100MHz  
  input rst,  
  input [3:0] data_in,  
  input load_data,  
  ///output  
  output reg spi_clk, //10MHz  
  output reg done_send,  
  output reg spi_data  
);  
  
  reg [2:0] dataCount;  
  reg [7:0] shiftReg;  
  reg [1:0] state;  
  reg [1:0] next_state;  
  wire clock_10;  
  reg CE;  
  
  clk_wiz_1(.clk_in1(clk), .reset(rst), .clk_out1(clock_10));
```

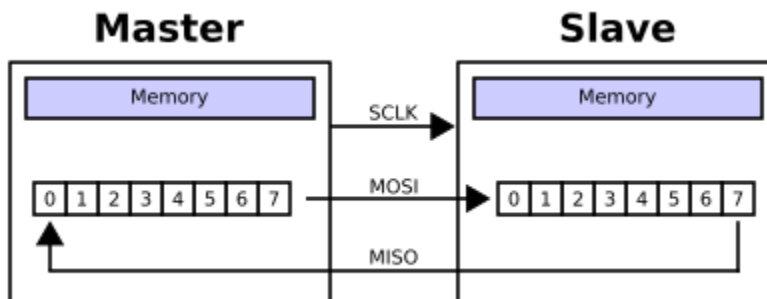
在 spiControl_FSM 的簡略狀態機設計如下:



IDLE state 下若沒有收到 load_data 訊號則一直在 IDLE state 下等待，若有 load_data 訊號就把輸入訊號存起來到 ShiftReg 內並且轉換到 TRANSFER state，在 TRANSFER state 下透過 dataCount 這個記數器判斷輸出的訊號 bit 數幾個了，若 dataCount 等於 7 時，代表 8bit 的 ShiftReg 內資料已經完全輸出並且將轉換到 DONE state 下，若 load_data 一直是拉高的則不會回到 IDLE state。下圖為此狀態機的 verilog code。

```
always @(*) begin //next_state
    case(state)
        IDLE:begin
            if(load_data) next_state = TRANSFER;
            else next_state = IDLE;
        end
        TRANSFER:begin
            if(dataCount == 3'd7) next_state = DONE;
            else next_state = TRANSFER;
        end
        DONE:begin
            if(!load_data) next_state = IDLE;
            else next_state = DONE;
        end
        default: next_state = IDLE;
    endcase
end
```

SPI master 端和 receiver 端內部皆使用到 Shift Register 來當作接收或傳遞時的 Buffer，因此此次設計 master 內部佔存資料的部分也是使用 Shift Register 來做實作。



```
always @(negedge clock_10 or posedge rst) begin //shiftReg
    if(rst) shiftReg <= 8'd0;
    else begin
        case(state)
            IDLE: if(load_data) shiftReg <= {data_in[3], 1'b0, data_in[2], 1'b0, data_in[1], 1'b0, data_in[0], 1'b0};
            TRANSFER: shiftReg <= {shiftReg[6:0],1'b0};
            DONE: shiftReg <= 8'd0;
        endcase
    end
end
```

```
always @(negedge clock_10 or posedge rst) begin //spi_data
    if(rst) spi_data <= 1'b0;
    else begin
        case(state)
            TRANSFER: spi_data <= shiftReg[7];
            default: spi_data <= 1'b0;
        endcase
    end
end
```

上圖分別為 Shift register 和 spi_data output 的 verilog code，IDLE state 下，若 load_data 訊號被拉高，shiftReg 將按鈕的 input(data_in)輸入進去，在 TRANSFER state 下則每個 negative edge 時將 ShiftReg 的 MSB 透過 spi_data 輸出出去並且向左移位將輸出值覆蓋掉。

spi_clk 輸出 clock 跟 spi_data 資料的擺放在 SPI mode 0 的情況下為，SPI master 在閒置時的 SPI_CLK 必須為 Low，並且在傳輸資料的時候，Master 端需要在 SPI_CLK 的 Negative Edge 時設置資料讓 Receiver 在第一個 SPI_CLK 為 Positive Edge 時就可以讀取資料。因此我將 spiControl_FSM 內部的所有 Sequential Logic 皆已 Negative Edge clock_10 的方式來實作，所以電路中的所有操作皆在 clock_10 這個訊號負緣時才會存入 Register 中，並且透過 CE(clock enable)訊號來決定 spi_clk 何時需要輸出，若沒輸出訊號則保持為 Low。下圖為 spi_clk 輸出訊號的 verilog code。

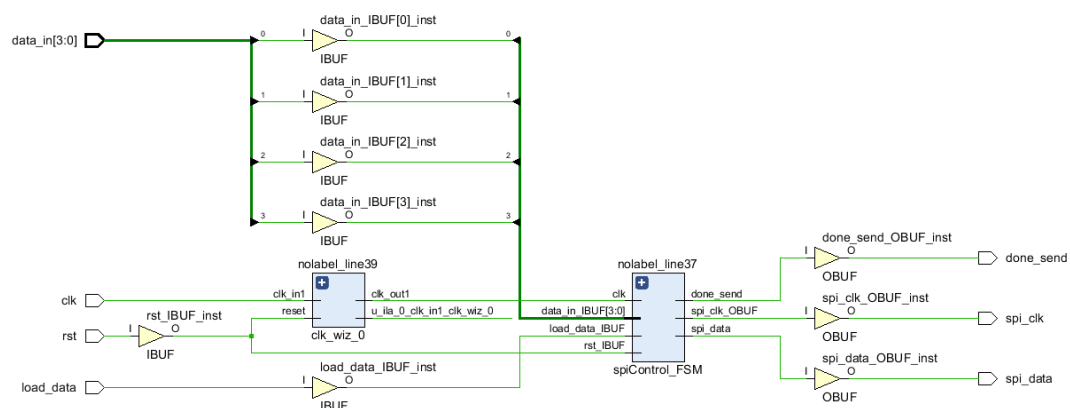
```
always @(negedge clock_10 or posedge rst) begin //CE
    if(rst) CE <= 1'b0;
    else begin
        case(state)
            TRANSFER: CE <= 1'b1;
            default: CE <= 1'b0;
        endcase
    end
end

always @(*) begin //spi_clk
    if(CE) spi_clk = clock_10;
    else spi_clk = 1'b0;
end
```

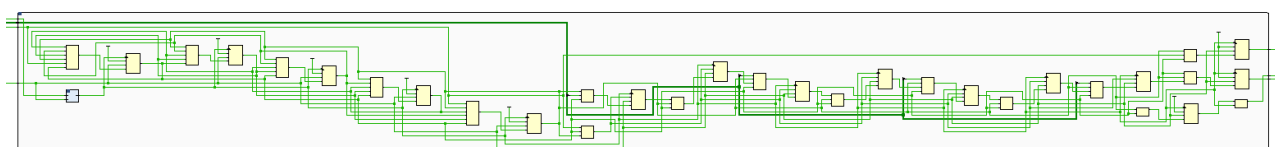
spiController_FSM 當中透過 dataCount 來計算輸出出去了幾筆 data 從 0 開始計算，並且也是使用 negative edge clock10 的方式來設計 sequentail circuit，在 TRANSFER state 下每經過一個 falling edge，spi_data 就會輸出一筆 data 此時 dataCount 就做加 1 的動作直到 dataCount 等於 7 時將會轉換到其他 state，在其他 state 時則將 dataCount 歸零。

```
always @(negedge clock_10 or posedge rst) begin //dataCount
    if(rst) dataCount <= 0;
    else begin
        case(state)
            TRANSFER: if(dataCount != 7) dataCount <= dataCount + 3'd1;
            default: dataCount <= 3'd0;
        endcase
    end
end
```

下圖為 Synthesis 後 Top Module 的 Schematic 表現圖:

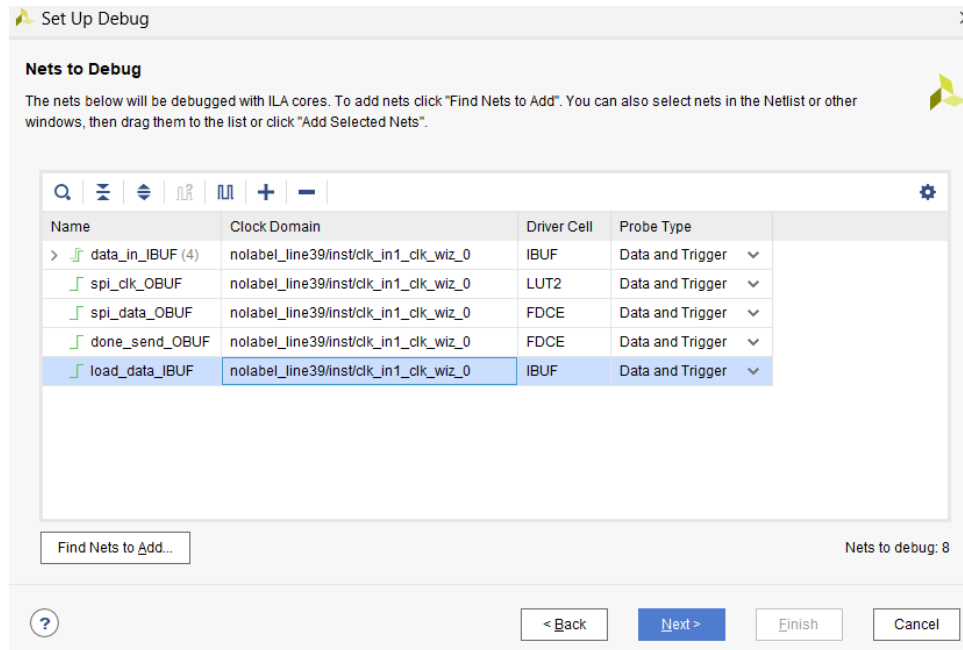


下圖為 spiControl_FSM 展開内部的 Schematic:

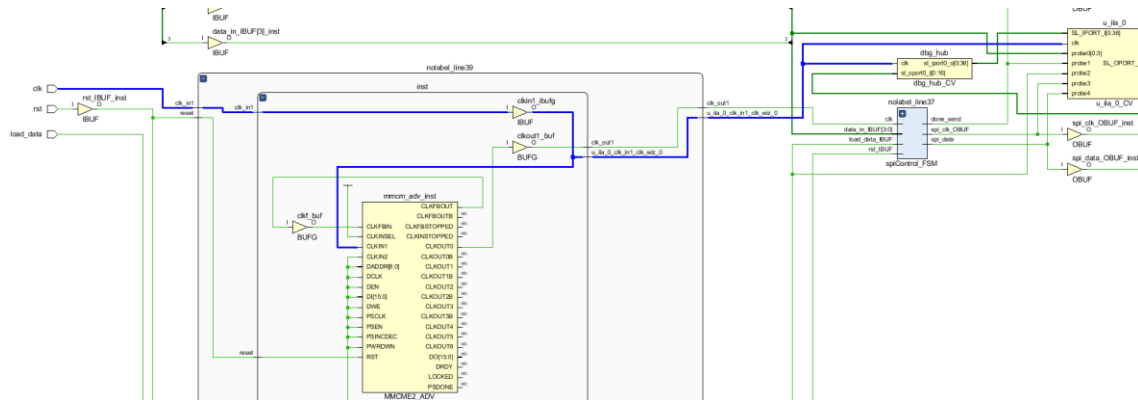


Part2: ILA core 軟體設定

設定 ILA core 要選擇哪寫線進行 output 呈現時，因 ILA 本身對於 clock 有嚴格規定必須為 free running clock，因此在選擇 clock domain 時必須選擇板子上 H16 pin 出來的 clock 才行，不能使用該訊實際所跑的那個 clock 訊號，因該 clock(125MHz)訊號為採樣訊號因此比要採集的訊號的 clock(10MHz)快是沒有關係的。

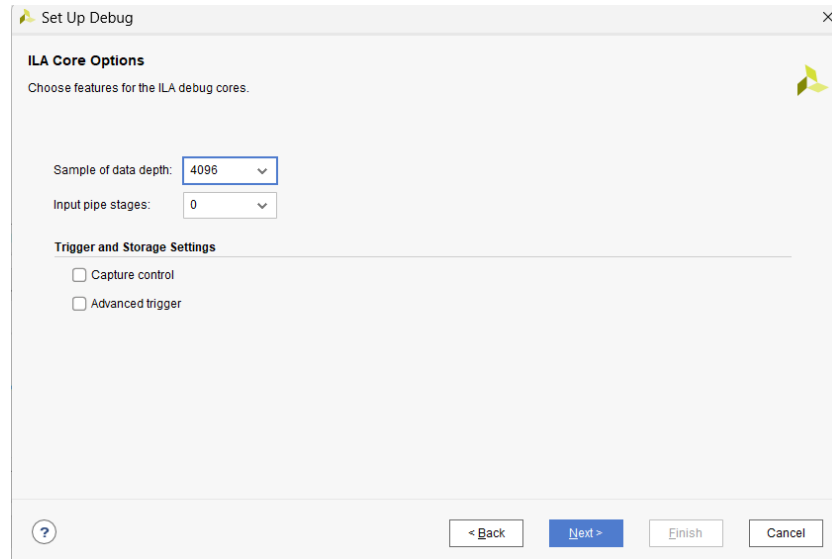


ILA Core 所選擇的 clk signal:

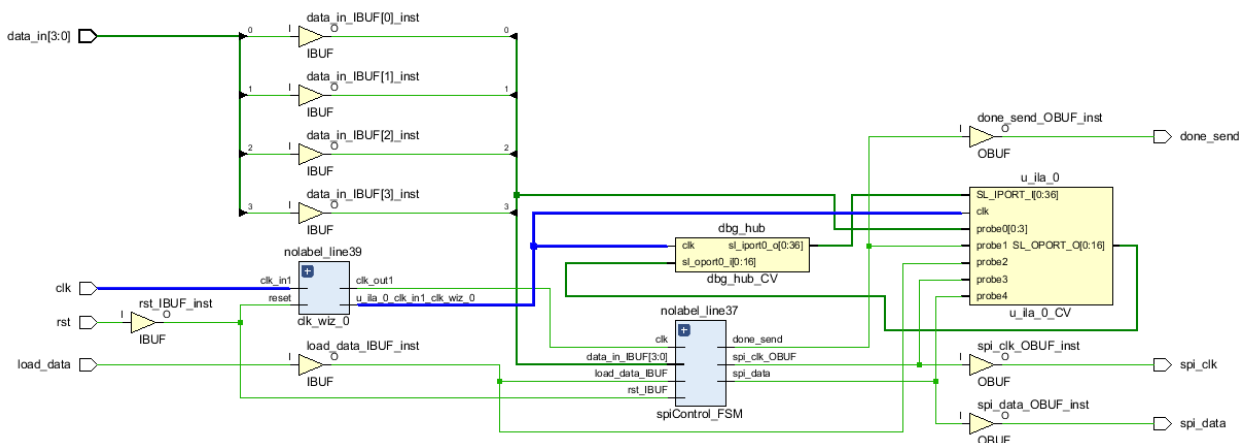


ILA Core Sample of data depth:

在這邊設置 ILA Core 的 Sample of data depth 的部分我選擇 4096，Sample of data depth 主要是 ILA Core 所採集的每條訊號線，每筆訊號他可以記下多少筆取樣的資料，也就是擷取的波形長度，以我的設計為例，我給予 ILA Core 的 clk 為 125MHz，代表 ILA Core 每條訊號線每秒可以擷取 125M 筆數值，而 depth 為 4096 代表我每條訊號線只能存下 4096 筆 0/1 的訊號，設定的 depth 越長所使用的 BRAM 資源將會越多，因 ILA Core implement 時是使用 FPGA 內部的 BRAM。



下圖為加入 ILA Core 後的 Schematic:



Part3: ILA Core 硬體設定實際操作

ILA Core 經軟體設定過後在 XDC 新增的一些指令:

1. 設定 ILA 的 Clock (資料擷取 Clock)

```
connect_debug_port u_ila_0/clk [get_nets [list nolabel_line38/inst/clk_in1_clk_wiz_0]]
connect_debug_port dbg_hub/clk [get_nets u_ila_0_clk_in1_clk_wiz_0]
```

- 第一行將 ILA 的擷取時鐘 (u_ila_0/clk) 接到 clock wizard 的輸出
- 第二行將 Debug Hub (JTAG 溝通用) 也接到同一顆 Clock

2. 建立 ILA 核心, 設定參數

```
create_debug_core u_ila_0 ila
set_property ALL_PROBE_SAME_MU true [get_debug_cores u_ila_0]
set_property ALL_PROBE_SAME_MU_CNT 1 [get_debug_cores u_ila_0]
```

- create_debug_core: 建立一個名叫 u_ila_0 的 ILA 核心
- ALL_PROBE_SAME_MU: 所有 probe 使用相同 Memory Unit, 資源較省
- MU_CNT 設成 1: 表示只使用一個資料儲存區

3. ILA 功能設定

```
set_property C_ADV_TRIGGER false [get_debug_cores u_ila_0]
set_property C_DATA_DEPTH 4096 [get_debug_cores u_ila_0]
set_property C_EN_STRG_QUAL false [get_debug_cores u_ila_0]
set_property C_INPUT_PIPE_STAGES 0 [get_debug_cores u_ila_0]
set_property C_TRIGIN_EN false [get_debug_cores u_ila_0]
set_property C_TRIGOUT_EN false [get_debug_cores u_ila_0]
```

- C_ADV_TRIGGER = false: 不啟用進階觸發功能
- C_DATA_DEPTH = 4096: 最多記錄 4096 筆樣本
- C_EN_STRG_QUAL = false: 不使用儲存過濾條件 (會記錄所有樣本)
- C_INPUT_PIPE_STAGES = 0: 輸入端不加 pipeline stage, 資料立即擷取
- C_TRIGIN_EN / C_TRIGOUT_EN = false: 不使用外部 Trigger pin (預設不需要)

4. 連接 Clock Port

```
set_property port_width 1 [get_debug_ports u_ila_0/clk]
connect_debug_port u_ila_0/clk [get_nets [list nolabel_line39/inst/clk_in1_clk_wiz_0]]
```

- port_width 1: 指定 clock 是單一位元
- 將 ILA clock 接到 clock wizard

5. 建立和連接 Probe (觀察的訊號)

```
set_property PROBE_TYPE DATA_AND_TRIGGER [get_debug_ports u_ila_0/probe0]
set_property port_width 4 [get_debug_ports u_ila_0/probe0]
connect_debug_port u_ila_0/probe0 [get_nets [list {data_in_IBUF[0]} {data_in_IBUF[1]} {data_in_IBUF[2]} {data_in_IBUF[3]}]]
create_debug_port u_ila_0 probe
```

- probe0 是一個 4-bit 的觀察訊號)
- DATA_AND_TRIGGER 表示這條訊號不只能觀察，也能作為觸發條件使用
- create_debug_port 表示還會再建立其他 probe (例如 probe1, probe2)

6. Debug Hub 設定 (控制 ILA 與 Vivado 的連線方式)

```
set_property C_CLK_INPUT_FREQ_HZ 300000000 [get_debug_cores dbg_hub]
set_property C_ENABLE_CLK_DIVIDER false [get_debug_cores dbg_hub]
set_property C_USER_SCAN_CHAIN 1 [get_debug_cores dbg_hub]
connect_debug_port dbg_hub/clk [get_nets u_ila_0_clk_in1_clk_wiz_0]
```

- C_CLK_INPUT_FREQ_HZ：指定 clock 頻率給 Debug Hub (不影響電路，但可協助 JTAG 傳輸穩定)
- C_ENABLE_CLK_DIVIDER = false：不使用 Clock divider
- C_USER_SCAN_CHAIN = 1：設定 scan chain ID
- connect_debug_port：把 clock 接到 Debug Hub，用來驅動 JTAG 傳輸

Capture Mode Settings:



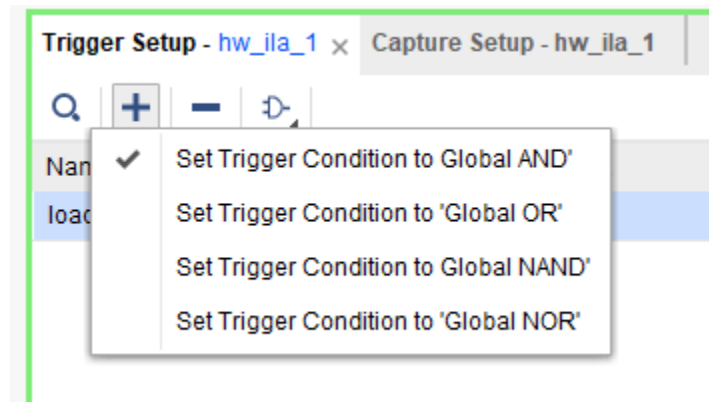
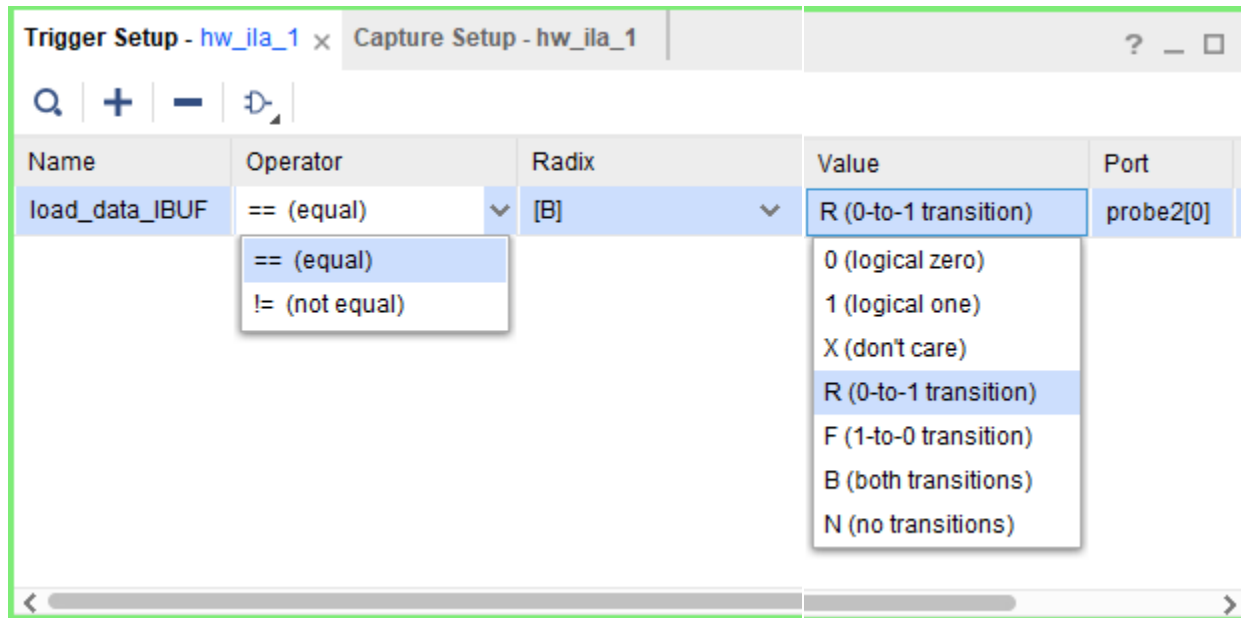
資料時間線（抓 4096 筆）：
 |<----- 前 1024 筆 ----->|<----- 後 3072 筆 ----->|
 ↑
 Trigger point（第 1024 筆）

ILA Status: Waiting For Trigger (1024 out of 4096 samples)

Name	Value
> data_in_IBUF[3:0]	0
done_send_OBUF	1
load_data_IBUF	1
spl_clk_OBUF	0
spl_data_OBUF	0

Updated at: 2025-Mar-22 19:19:42

Trigger Setup:



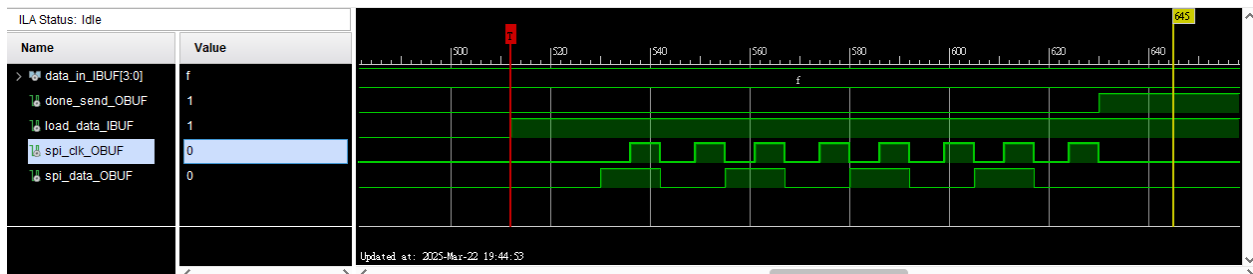
上方的設定代表:

當 load_data_IBUF 從 0 變成 1(Rising Edge) 的那一刻，觸發擷取波形資料，

Set Trigger Condition to Global AND 則代表若是有其他調觸發條件，則全部的觸發條件必須都發生才會擷取資料。

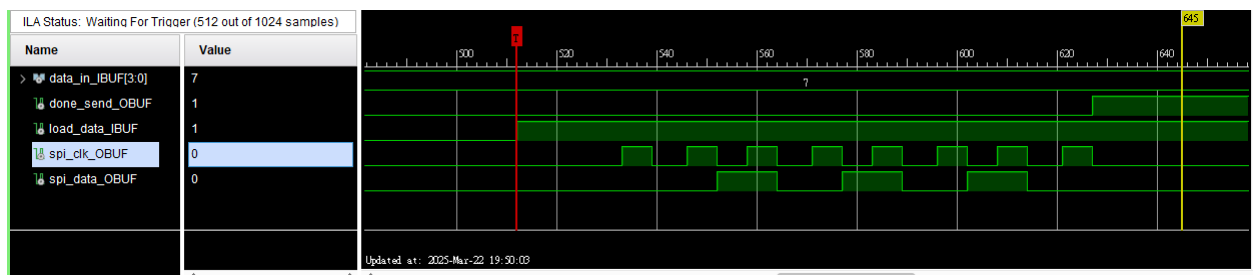
Waveform Verification:

data_in 為 1111 (Trigger: load_data rising edge):



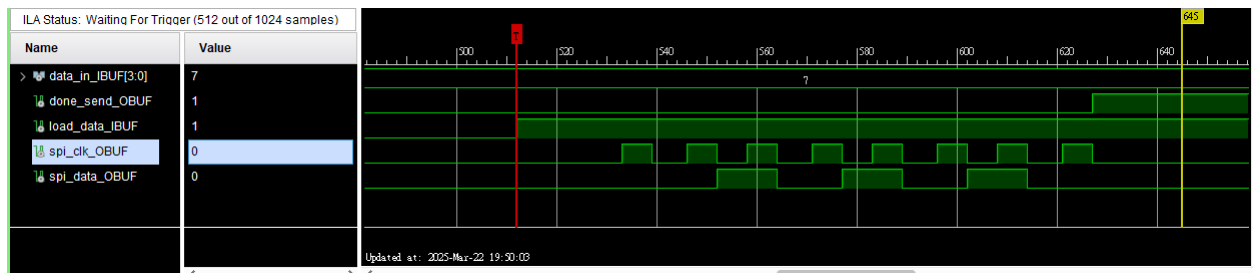
在 spiControl_FSM 設計時將 data_in 透過 zigzag 穿插 0 的方式放入 ShifReg 裡面，所以輸出波形時會變成輸出 10101010，從上圖可以看到我們設計的 Mode 0 spiControl_FSM 在 negative edge 時將 spi_data 輸出出去，讓接收端可以在第一個 positive edge 時讀取到資料。

data_in 為 0111 (Trigger: load_data rising edge):



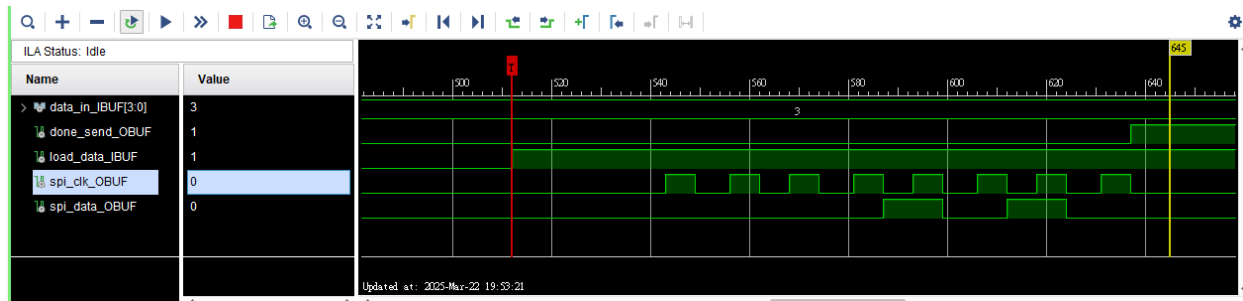
從 spi_data 可以看到輸出波型為 00101010。

data_in 為 0111 (Trigger: load_data rising edge):



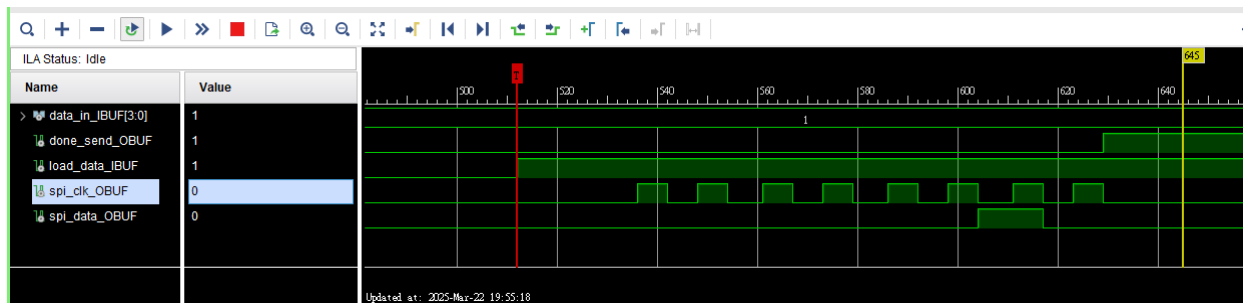
從 spi_data 可以看到輸出波型為 00101010。

data_in 為 0011 (Trigger: load_data rising edge):



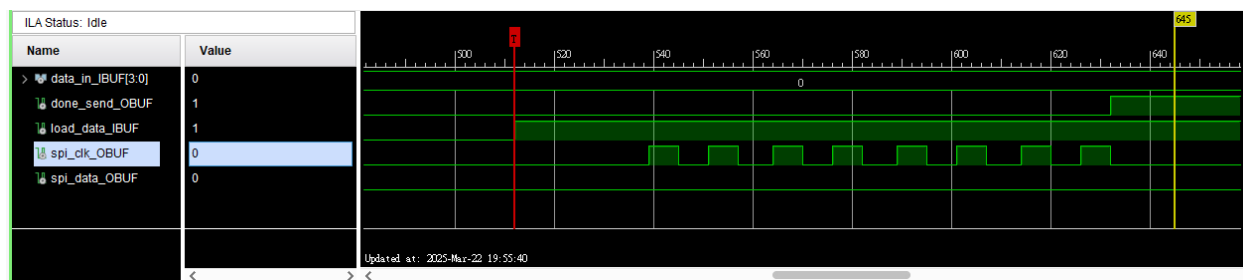
從 spi_data 可以看到輸出波型為 00001010。

data_in 為 0001 (Trigger: load_data rising edge):



從 spi_data 可以看到輸出波型為 00000010。

data_in 為 0001 (Trigger: load_data rising edge):



從 spi_data 可以看到輸出波型為 00000000。

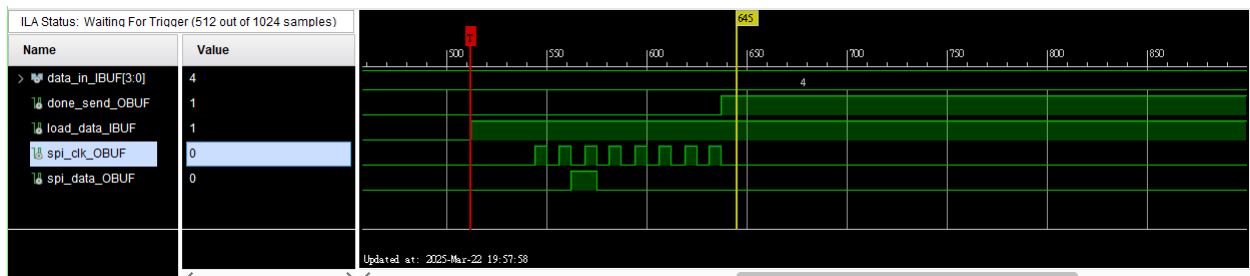
Trigger Setup(data_in = 0100 AND load data positive edge):

Trigger Setup - hw_ila_1 x Capture Setup - hw_ila_1 ? _ □

Q + - ↺

Name	Operator	Radix	Value	Port
data_in_IBUF[...]	==	[B]	0100	probe0[...]
load_data_IBUF	==	[B]	R	probe2[0]

< >



從 spl_data 可以看到輸出波型為 00100000。